INFORME DE AVANCE TÉCNICO Y EVIDENCIAS

INTEGRANTES DEL EQUIPO:
• Estudiante 1: Jorge Guzmán — GitHub: JorgeGuzman-32
• Estudiante 2: Francisco Órdenes — GitHub: Cisco-fbn
Asignatura: Desarrollo de Aplicaciones Móviles
Docente de Cátedra — Duoc UC

1. Introducción y Contexto del Proyecto

El presente informe de avance técnico tiene como objetivo evidenciar de forma estructurada, ordenada y verificable las funcionalidades clave y componentes desarrollados dentro de la aplicación móvil de gestión de postventa y reclamos para retail. La solución aplica de manera estricta el patrón arquitectónico MVVM, mecanismos de persistencia local reactiva mediante Room Database, validación avanzada de entradas de usuario, animaciones y acceso seguro a recursos nativos de hardware, cumpliendo rigurosamente con los indicadores de logro establecidos para la Evaluación Parcial 2 de la asignatura.

2. Registro y Fichas de Hitos de Desarrollo

Hito 1: Nombre	Secuencia Obligatoria de Navegación y Flujo Desacoplado
Descripción breve	Configuración e integración del componente NavHost global para forzar la secuencia de acceso lineal solicitada en los lineamientos (LoginScreen ➔ MenuScreen ➔ ListaSolicitudesScreen ➔ SolicitudFormScreen).

Evidencia visual (Código / Interfaz)



```
2 Usages
@Composable
fun AppNavigation(viewModel: SolicitudViewModel) {
    val navController = rememberNavController()


    NavHost(navController = navController, startDestination = "login") {

        // 1. Pantalla de Login requerida
        composable( route = "login") {
            LoginScreen(onLoginSuccess = {
                navController.navigate( route = "menu") {
                    popUpTo( route = "login") { inclusive = true }
                }
            })
        }

        // 2. Pantalla de Menú requerida
        composable( route = "menu") {
            MenuScreen(onNavigateToSolicitudes = {
                navController.navigate( route = "lista_solicitudes")
            })
        }

        // 3. Pantalla de lista (Adaptada al flujo)
        composable( route = "lista_solicitudes") {
            ListaSolicitudesScreen(
                onNuevaSolicitudClick = { navController.navigate( route = "formulario_solicitud") },
                onSolicitudClick = { id -> navController.navigate( route = "detalle_solicitud/$id") }
            )
        }

        // 4. Pantalla de Formulario
        composable( route = "formulario_solicitud") {
            SolicitudFormScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

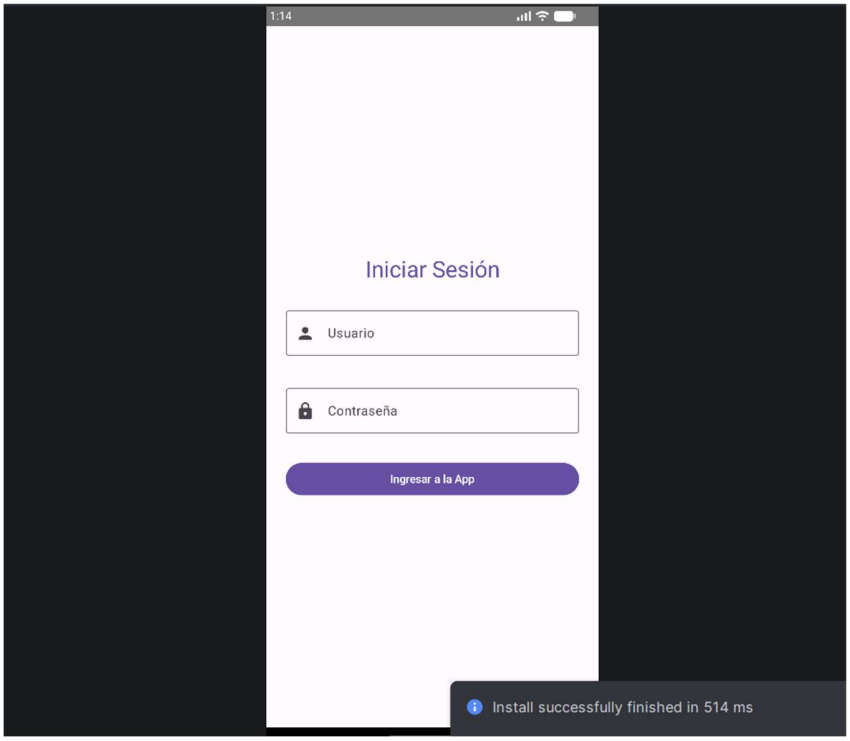
        // 5. Pantalla de Detalle conservada intacta
        composable(
            route = "detalle_solicitud/{id}",
            arguments = listOf(navArgument( name = "id") { type = NavType.IntType })
        ) { backStackEntry ->
            val id = backStackEntry.arguments?.getInt( key = "id") ?: 0
            SolicitudDetalleScreen(
                solicitudId = id,
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }
```

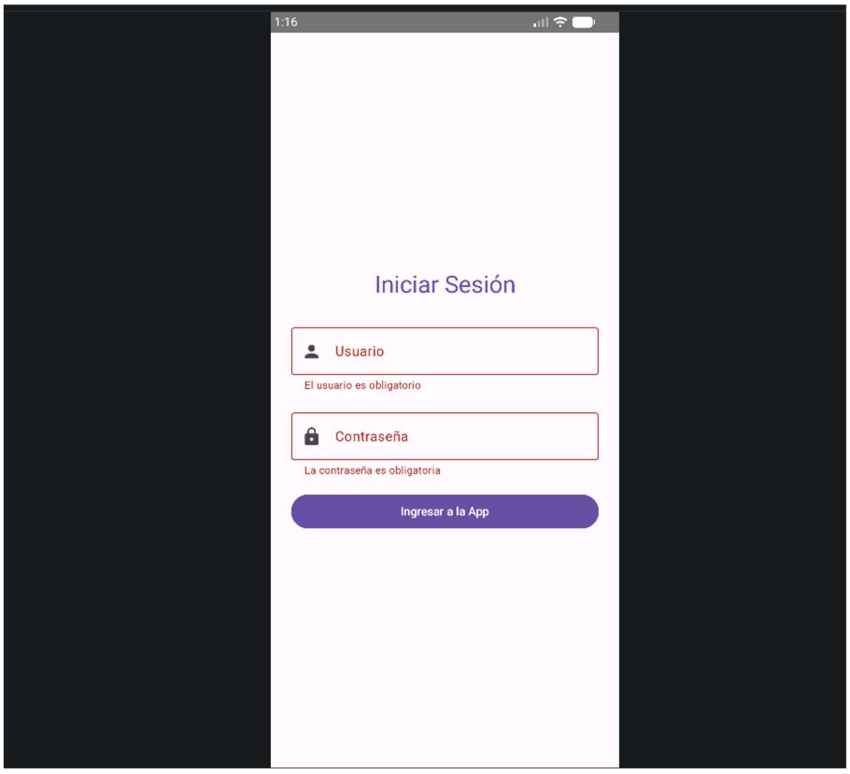
Estructura lógica aplicada en el archivo:

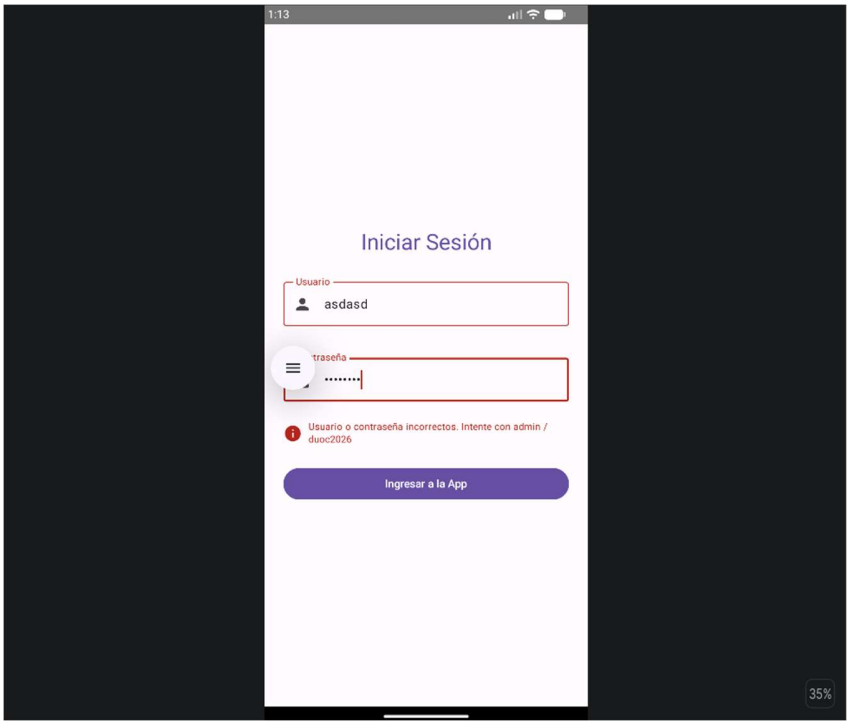
```
NavHost(navController = navController, startDestination = "login") {
    composable("login") { LoginScreen(onLoginSuccess = {
navController.navigate("menu") }) }
    composable("menu") { MenuScreen(onNavigateToSolicitudes = {
navController.navigate("lista_solicitudes") }) }
}
```

Resultado obtenido

Se comprobó el correcto funcionamiento ejecutando la aplicación desde

	cero en el emulador, verificando que la vista de arranque inicial sea obligatoriamente el LoginScreen, impidiendo saltos de seguridad manuales.
Hito 2: Nombre	Interfaz de Acceso Segura con Validación Local de Credenciales
Descripción breve	Diseño de la vista LoginScreen utilizando componentes tipográficos de Material Design 3, agregando cajas controladas para Usuario y Contraseña que implementan máscaras visuales seguras.
Evidencia visual (Código / Interfaz)	


1:16
Iniciar Sesión
Usuario
El usuario es obligatorio
Contraseña
La contraseña es obligatoria
Ingresar a la App


1:13
Iniciar Sesión
Usuario
asdasd
Usuario o contraseña incorrectos. Intente con admin / duoc2026
Ingresar a la App
35%

Estructura lógica aplicada en el archivo:

```
OutlinedTextField(
    value = contrasena, onValueChange = { contrasena = it },
    visualTransformation = PasswordVisualTransformation(),
    isError = contrasenaError != null
)
```

Resultado obtenido

Se comprobó digitando datos inválidos intencionalmente. Al presionar el botón de acceso, la interfaz bloqueó la navegación y gatilló de forma reactiva las alertas correspondientes.

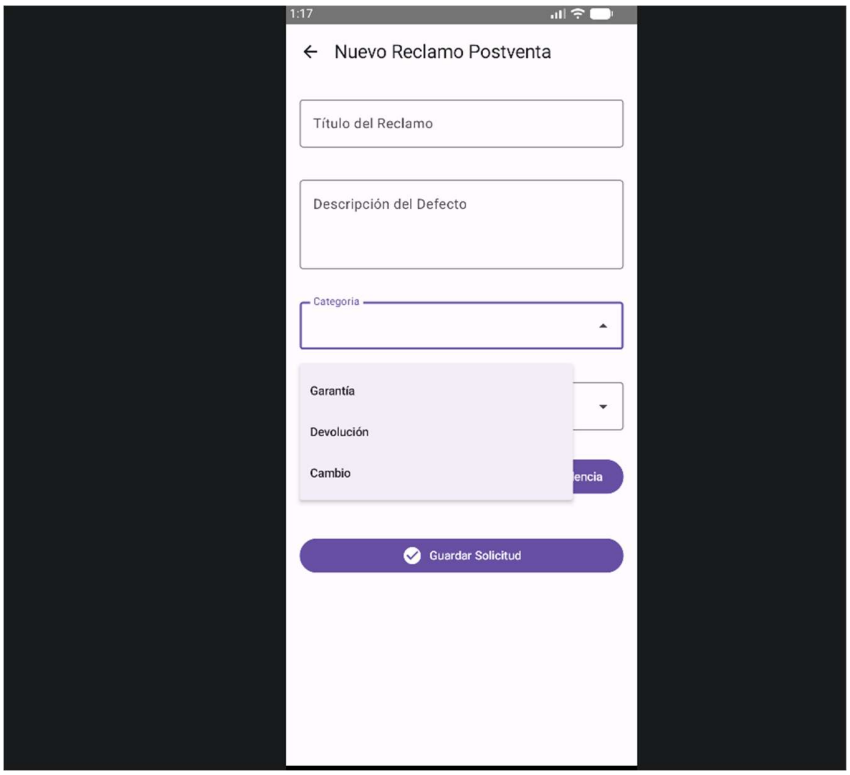
Hito 3: Nombre

Filtros Interactivos mediante Menús Desplegables de Selección Única

Descripción breve

Transformación de los campos de texto plano de Categoría y Servicio del formulario a componentes interactivos del sistema nativo ExposedDropdownMenuBox, obligando al usuario a seleccionar una opción válida de una lista.

Evidencia visual (Código / Interfaz)





Estructura lógica aplicada en el archivo:

ExposedDropdownMenuBox(expanded = expandido, onExpandedChange = { expandido = !expandido }) {
OutlinedTextField(value = state.categoria, readOnly = true, modifier = Modifier.menuAnchor())
ExposedDropdownMenu(...) { DropdownMenuItem(text = { Text(opcion) }, onClick = { ... }) }
}

Resultado obtenido

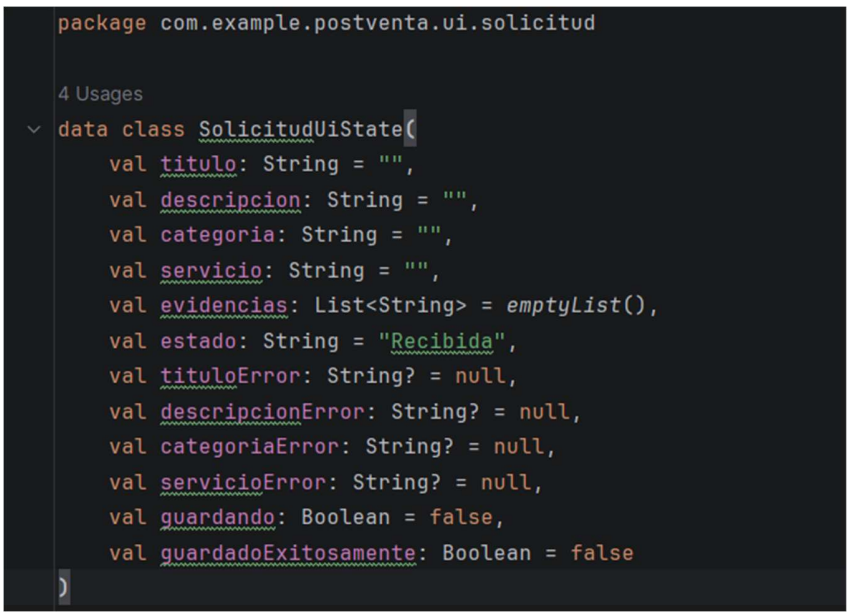
Se testeó haciendo clic sobre el recuadro. El teclado nativo se mantuvo oculto (readOnly = true), desplegó el menú en un hilo flotante y actualizó el estado al presionar una opción de la lista.

Hito 4: Nombre

Gestión de Estados Centralizada e Inmutable mediante StateFlow

Descripción breve

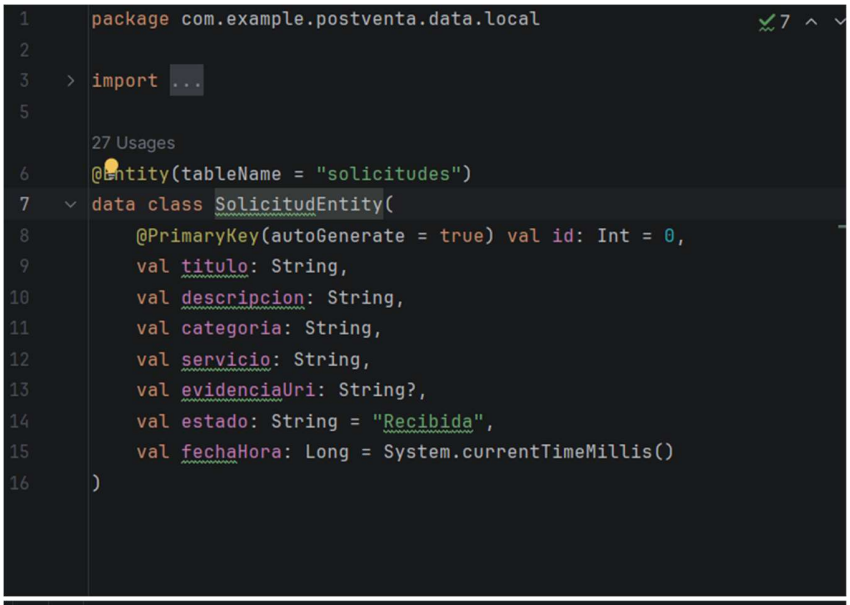
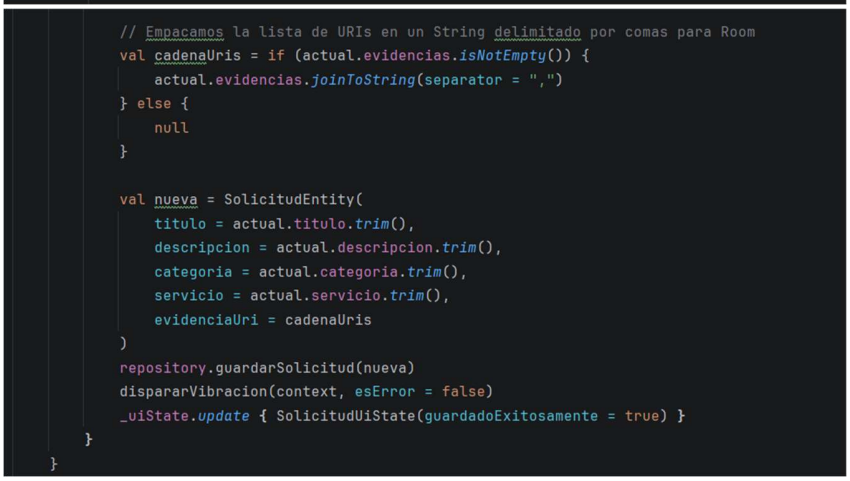
Implementación de la clase de datos estructurada SolicitudUiState para encapsular todas las variables primitivas del formulario, incluyendo los contenedores dedicados para la retención de mensajes de error.

Evidencia visual (Código / Interfaz)	 *Estructura lógica aplicada en el archivo:* data class SolicitudUiState(val titulo: String = "", val tituloError: String? = null, val guardando: Boolean = false)
Resultado obtenido	Se validó depurando el ciclo de vida de la aplicación con Logcat. El estado mutable interno cambia de forma segura y se emite de manera asíncrona hacia las vistas Compose mediante StateFlow.

Hito 5: Nombre	Validación Reactiva Desacoplada en Capa de Negocio (ViewModel)
Descripción breve	Programación de las reglas de control de strings (.isBlank()) dentro de SolicitudViewModel, aislando la lógica dura de control fuera de los componentes visuales para respetar la arquitectura MVVM.

Evidencia visual (Código / Interfaz)	 *Estructura lógica aplicada en el archivo:* val tErr = if (actual.titulo.isBlank()) "El título es obligatorio" else null if (tErr != null) { _uiState.update { it.copy(tituloError = tErr) }; return }
Resultado obtenido	Se verificó dejando los campos vacíos en el formulario y presionando el botón 'Guardar'. El ViewModel interceptó el flujo, disparó la inyección háptica y actualizó las banderas de error del estado.

Hito 6: Nombre	Almacenamiento Local Offline y Conversión Estructurada con Room
Descripción breve	Configuración del motor SQLite local mediante las abstracciones DAO y Entity de Room. Incluye la serialización de listas dinámicas de URIs de fotos comprimidas en cadenas delimitadas por comas para su persistencia.

Evidencia visual (Código / Interfaz)	  *Estructura lógica aplicada en el archivo:* @Entity(tableName = "solicitudes") data class SolicitudEntity(@PrimaryKey(autoGenerate = true) val id: Int = 0, val titulo: String, val evidenciaUri: String?)
Resultado obtenido	Se comprobó guardando un reclamo con éxito, cerrando la aplicación por completo desde el administrador de tareas del dispositivo y volviendo a ingresar; los registros persistieron intactos en la lista.
Hito 7: Nombre	Animaciones Funcionales y Retroalimentación de Error Dinámica

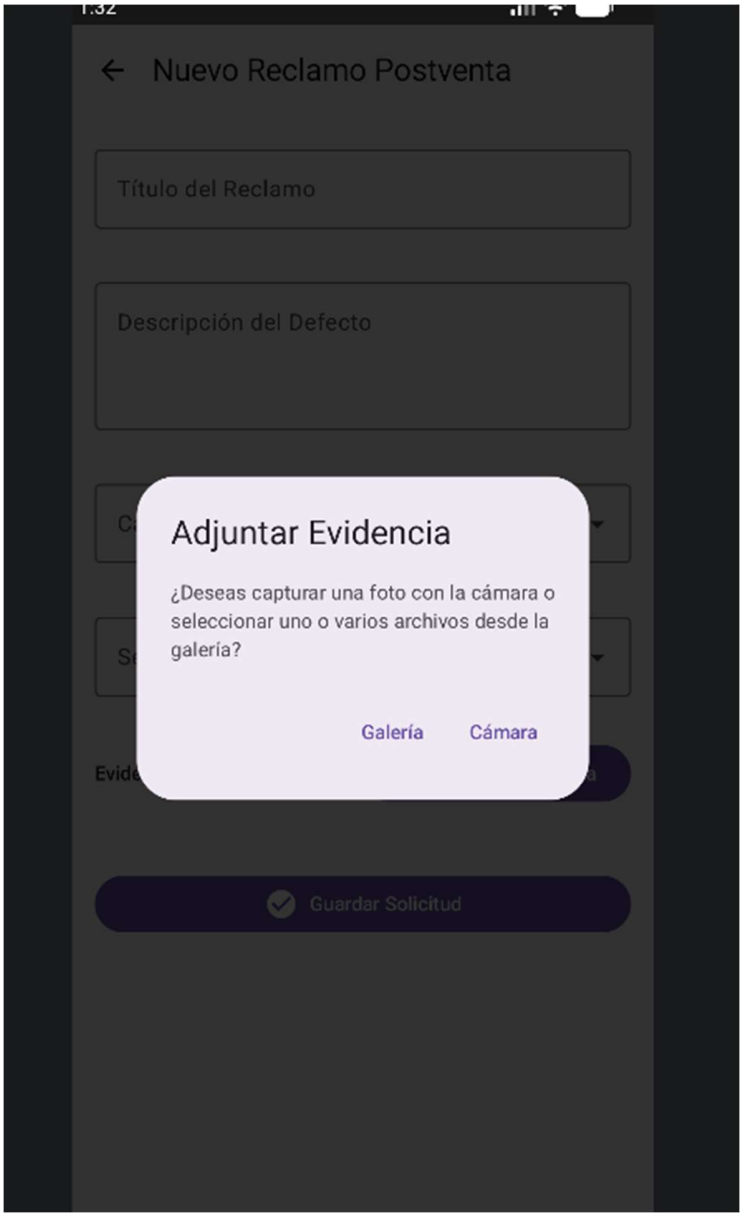
Descripción breve	Inserción de bloques animados mediante AnimatedVisibility integrados con transiciones de desvanecimiento y expansión vertical para la aparición controlada y orgánica de los textos informativos de error.

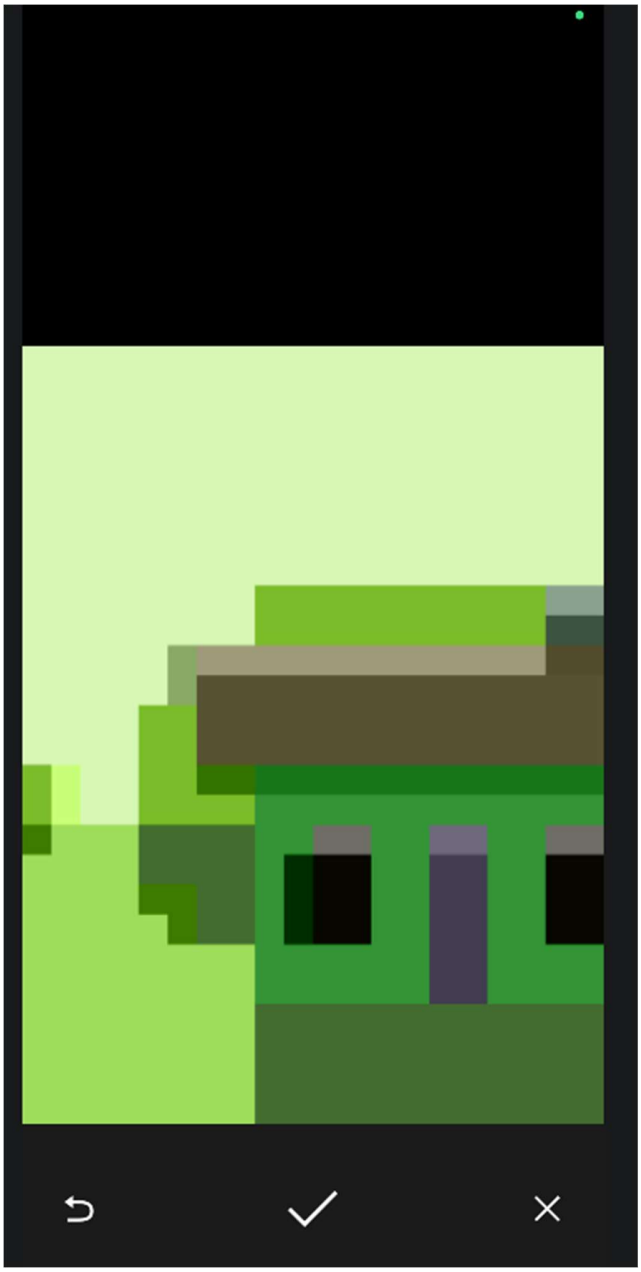
Evidencia visual (Código / Interfaz)	
	Estructura lógica aplicada en el archivo:
	AnimatedVisibility(visible = state.tituloError != null, enter = fadeIn() + expandVertically()) {
	Text(text = state.tituloError ?: "", color = MaterialTheme.colorScheme.error)
	}

Resultado obtenido	Se comprobó visualmente al gatillar un error en el formulario; los textos instructivos no aparecieron de golpe, sino que se desplazaron suavemente hacia abajo en la pantalla con fluidez.

Hito 8: Nombre	Acceso Seguro a Recursos de Hardware Nativo (Cámara y Vibración)
Descripción breve	Integración del contrato asíncronos nativo TakePicturePreview para interactuar con la cámara física y uso del servicio inyectado VIBRATOR_SERVICE para emitir vibraciones hápticas diferenciadas ante éxitos y fallas.

Evidencia visual (Código / Interfaz)





Estructura lógica aplicada en el archivo:

```
val cameraLauncher =
rememberLauncherForActivityResult(ActivityResultContracts.TakePictur
ePreview()) { bitmap ->
    bitmap?.let { val uri = bitmapToUri(context, it);
viewModel.agregarEvidencias(listOf(uri.toString())) }
}
```

Resultado obtenido

Se comprobó capturando una foto real desde el formulario y

adjuntándola al carrusel, y verificando físicamente la vibración prolongada en el dispositivo al dispararse un error visual.